

INDUSTRY PROBLEM SOLVING

CO4 - AT2

1. A telecommunications company wants to recommend suitable data plans to customers based on their usage patterns. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity between users is measured, how recommendations are generated, and how the system adapts to changing user behavior over time.

Input

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Data: [Data GB, Calls min, SMS, Streaming hrs]
X = np.array([
    [2,100,50,5],
    [5,200,80,15],
    [10,300,100,30],
    [3,120,60,8],
    [8,250,90,25],
    [4,150,70,10],
    [9,280,95,28],
    [2.5,110,55,6]
])

# 0 = Basic, 1 = Standard, 2 = Premium
y = np.array([0,1,2,0,2,1,2,0])

# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KNN model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)

# Test customer
customer = np.array([[6,180,70,18]])
customer_scaled = scaler.transform(customer)
```

Output

```
Nearest Customers:  
Customer 2 Distance = 0.774  
Customer 6 Distance = 1.147  
Customer 4 Distance = 1.755  
  
New Customer Usage: [ 6 180 70 18]  
Recommended Plan: Standard  
Model Accuracy: 100.00%
```

2. An energy management company aims to predict electricity consumption based on factors such as time, weather, and appliance usage. Propose a solution using Locally Weighted Regression or RBF networks. Explain how local learning captures variations in consumption patterns and how the model handles noisy and seasonal data.

Input

```
import numpy as np
X = np.array([
    [8, 22, 3],
    [9, 24, 4],
    [10, 26, 5],
    [12, 30, 6],
    [14, 32, 7],
    [16, 31, 6],
    [18, 28, 8],
    [20, 25, 7],
    [22, 23, 5],
    [24, 21, 3]
], dtype=float)

# Electricity consumption (kWh)
y = np.array([2.1, 2.5, 3.0, 4.2, 4.8,
              4.5, 5.2, 4.6, 3.5, 2.7])

# Normalize input data
X_min = X.min(axis=0)
X_max = X.max(axis=0)
Xn = (X - X_min) / (X_max - X_min)

def LWR(x_query, tau=0.4):
    # Normalize query
    xq = (x_query - X_min) / (X_max - X_min)

    # Calculate Gaussian weights
    distances = np.sum((Xn - xq) ** 2, axis=1)
    weights = np.exp(-distances / (2 * tau ** 2))

    # Add bias
    A = np.c_[np.ones(len(Xn)), Xn]
    q = np.r_[1, xq]
```

Output:

```
Test Input [Time, Temperature, Appliances]: [19 27 7]

Local Weights:
[0.016 0.076 0.22 0.385 0.386 0.523 0.85 0.891 0.359 0.039]

Predicted Electricity Consumption: 4.71 kWh

Effect of Tau:
Tau = 0.1 -> Prediction = 4.73 kWh
Tau = 0.3 -> Prediction = 4.71 kWh
Tau = 0.5 -> Prediction = 4.7 kWh
Tau = 1.0 -> Prediction = 4.68 kWh
```